

JavaScript Functions and Classes

Student Name: P. Dharani	Roll No.: A24126552261
Course: Full Stack Web Development	Course Code: 23CM4121
Experiment No.: 5	

① Title

Differentiate Between JavaScript Functions and Classes and Create Multiple Objects Using a Class

② Aim

To understand the difference between JavaScript functions and classes and to demonstrate how a class can be used to create multiple objects with common properties and methods.

③ Objective

- To understand the purpose of JavaScript functions.
- To understand classes, objects, properties, constructors and methods.
- To create multiple objects from a single JavaScript class.
- To use DOM manipulation and an event listener to display object details.
- To observe the common structure and different data of objects created from the same class.

④ Theory

JavaScript Function: A function is a reusable block of code written to perform a particular task. It can accept parameters and return a value.

JavaScript Class: A class is a blueprint for creating objects. It defines the common properties and methods that all its objects can use.

Constructor: A special method that runs automatically when an object is created with the `new` keyword. It sets the initial property values of the object.

Object: An object is an instance of a class. Many objects can be created from one class, and each object keeps its own property values.

Point	Function	Class
Purpose	Performs a task and may return a value.	Acts as a blueprint to create objects.
Called / created with	Function call, e.g. <code>greetStudent("Teacher")</code> .	The <code>new</code> keyword, e.g. <code>new Student(...)</code> .
Data storage	Uses parameters and local variables.	Stores data as properties using <code>this</code> .
Reuse	Reuses a block of logic.	Reuses both data structure and methods.

In this experiment a normal function `greetStudent()` is used together with a `Student` class. Three `Student` objects are created from the same class; each has the properties `name`, `course` and `year` and uses the common `getDetails()` method.

Basic flow: Browser → JavaScript Event → Class Objects → DOM Manipulation → Displayed Student Details

⑤ Algorithm / Procedure

1. Create an HTML page containing a heading, description, button and output area.
2. Create a function named `greetStudent()` to demonstrate a normal function.
3. Define a `Student` class with a constructor.

4. Initialize the student name, course and year inside the constructor.
5. Define the `getDetails()` method to return the student's details.
6. Create three Student objects using the `new` keyword.
7. Select the output element using DOM methods.
8. Attach a click event listener to the button.
9. When the button is clicked, display the greeting and all three Student objects.
10. Open the HTML file in a browser and verify the output.
11. Test the page before and after clicking the button.

6 Program

Folder structure

```
Q5-JavaScript-Functions-and-Classes/  
├── src/  
│   ├── index.html  
│   ├── script.js  
│   └── style.css
```

index.html

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8">  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  <title>Functions and Classes Demo</title>  
  <link rel="stylesheet" href="style.css">  
</head>  
<body>  
  <div class="card">  
    <h1>Functions vs Classes in JavaScript</h1>  
    <p class="intro">  
      A normal function and a Student class are used below to build  
      three student objects.  
    </p>  
    <button id="loadBtn">Display Students</button>  
    <div id="result"></div>  
  </div>  
  <script src="script.js"></script>  
</body>  
</html>
```

script.js

```
// ----- Normal function -----  
function greetStudent(name) {  
  return "Hello, " + name + "! Here are the student records.";  
}  
  
// ----- Class (blueprint) -----  
class Student {  
  constructor(name, course, year) {  
    this.name = name;  
    this.course = course;  
    this.year = year;  
  }  
  
  // Common method shared by every Student object  
  getDetails() {  
    return "Course: " + this.course + " | Year: " + this.year;  
  }  
}  
  
// ----- Three objects from the same class -----
```

```

const student1 = new Student("Aarav", "IT", "1st Year");
const student2 = new Student("Diya", "ECE", "2nd Year");
const student3 = new Student("Farhan", "CSE", "4th Year");
const allStudents = [student1, student2, student3];

// ----- DOM and event handling -----
const resultBox = document.getElementById("result");
const loadBtn = document.getElementById("loadBtn");

function showStudents() {
    resultBox.innerHTML = "";           // clear old output first

    const greeting = document.createElement("h2");
    greeting.textContent = greetStudent("Teacher");
    resultBox.appendChild(greeting);

    for (const s of allStudents) {
        const box = document.createElement("div");
        box.className = "student-card";

        const nameTag = document.createElement("h3");
        nameTag.textContent = s.name;

        const details = document.createElement("p");
        details.textContent = s.getDetails();

        box.appendChild(nameTag);
        box.appendChild(details);
        resultBox.appendChild(box);
    }
}

loadBtn.addEventListener("click", showStudents);

```

style.css

```

body {
    font-family: Arial, sans-serif;
    background-color: #ffffff;
    margin: 0;
    padding: 30px;
}

.card {
    width: 540px;
    padding: 22px;
    border: 1px solid #999999;
}

h1 {
    font-size: 24px;
    margin-top: 0;
}

button {
    padding: 8px 16px;
    cursor: pointer;
}

.student-card {
    margin-top: 12px;
    padding: 10px 14px;
    border: 1px solid #cccccc;
    border-left: 5px solid #000000;
}

.student-card h3 {
    margin: 0 0 6px 0;
}

```

```
.student-card p {
  margin: 3px 0;
}
```

7 Code Explanation

Code / Concept	Explanation
function greetStudent(name)	A normal function. It takes a name as a parameter and returns a greeting string; it can be called any number of times.
class Student	Defines the blueprint (class) from which Student objects are created.
constructor(name, course, year)	Special method that runs automatically when <code>new Student(...)</code> is used; it receives the values for the new object.
this.name, this.course, this.year	Properties that store the values belonging to the current object. Every object has the same property names but its own values.
getDetails()	Common class method shared by all objects; it returns the course and year of the student as a text string.
new Student("Aarav", "IT", "1st Year")	Creates an object (instance) of the class. Three objects (student1, student2, student3) are created with different data.
const allStudents = [...]	Keeps the three objects in an array so that they can be displayed with a loop.
document.getElementById()	Selects the output area (<code>result</code>) and the button from the HTML page.
document.createElement(), appendChild()	Create new heading, paragraph and card elements and add them to the output area (DOM manipulation).
resultBox.innerHTML = ""	Clears the old output first, so clicking the button again does not repeat the cards.
addEventListener("click", showStudents)	Runs the <code>showStudents</code> function whenever the button is clicked.

8 Test Cases and Execution Data

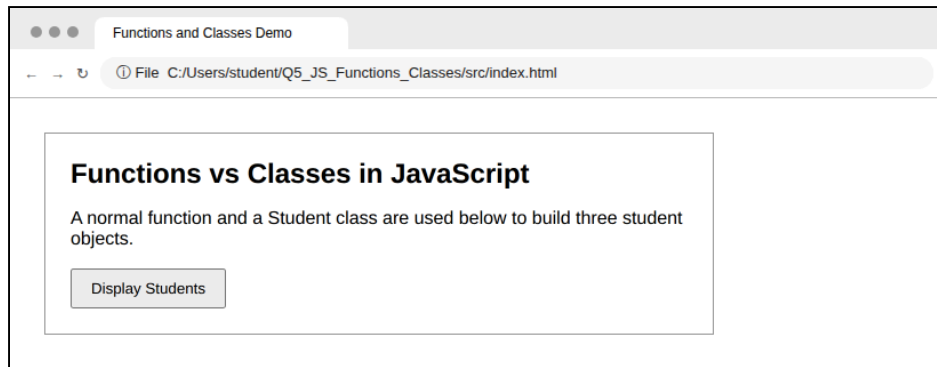
The page was tested in a browser (Chromium) before and after clicking the button. This experiment accepts no user-entered form data, so invalid-input validation is not applicable.

Test Case	Action	Expected Result	Actual Result	Status
TC-01	Open index.html	Page loads with heading, description and the "Display Students" button; the output area is empty.	Heading "Functions vs Classes in JavaScript" and button "Display Students" shown; 0 student cards present.	Pass
TC-02	Click "Display Students"	Greeting "Hello, Teacher! Here are the student records." and three student cards are displayed.	Greeting "Hello, Teacher! Here are the student records."; 3 cards: Aarav, Diya, Farhan.	Pass
TC-03	Check details of each object	Aarav: IT, 1st Year; Diya: ECE, 2nd Year; Farhan: CSE, 4th Year.	Aarav: IT, 1st Year; Diya: ECE, 2nd Year; Farhan: CSE, 4th Year.	Pass
TC-04	Check class and function (browser console)	<code>typeof greetStudent</code> is a function; all three objects are instances of Student.	<code>greetStudent</code> : function; <code>student1</code> , <code>student2</code> , <code>student3</code> instanceof Student: true; they share one prototype: true.	Pass
TC-05	Click the button a second time (special case)	No error; still exactly three cards and one greeting (no duplicates).	3 cards and 1 greeting after second click; console errors: 0.	Pass

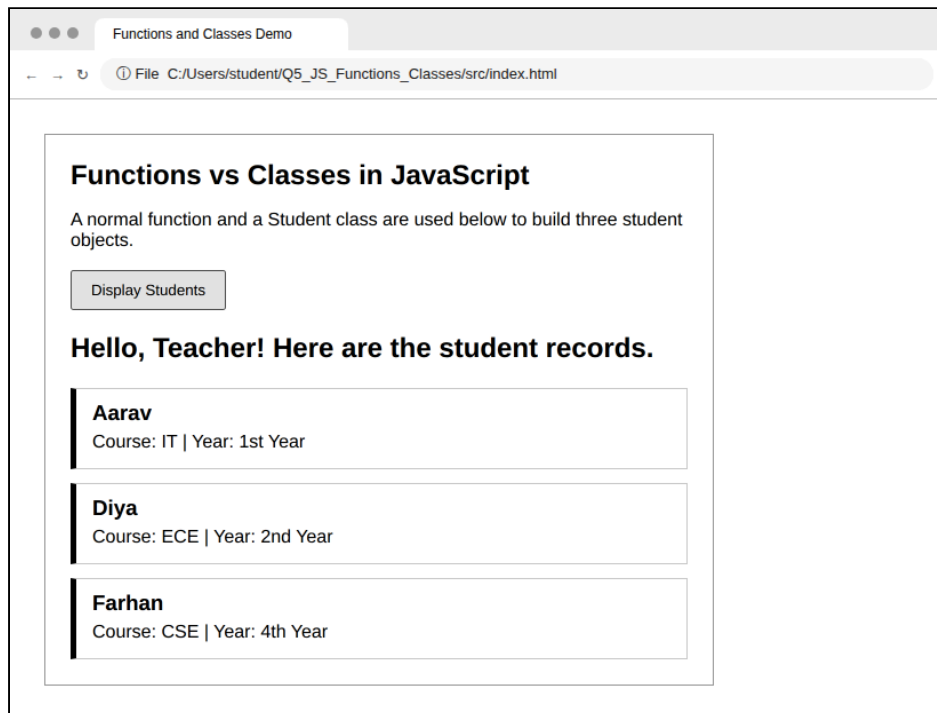
Validation: The page was checked before and after the click. The second state confirmed that the three objects created from one class share the common `getDetails()` method while holding different data.

9 Output

Output 1: Initial webpage (before clicking the button)



Output 2: After clicking "Display Students"



10 Result

The JavaScript Functions and Classes experiment was implemented successfully. A normal function was demonstrated, a Student class was created, and multiple Student objects were instantiated with common properties and methods. The page was executed in a browser and the displayed output verified the working of the program.

End of Observation Record